Lesson 5: Enemy Ranks

TIC-80

VIDEO GAME CREATION

This week: one enemy becomes many. Your job is to turn last week's single enemy into a whole row of them, using a loop instead of repeating yourself by hand.

Open your own [YourName]Game project from Lesson 4 - don't start a new one. Everything this week builds on the enemy and collision detection you already have.

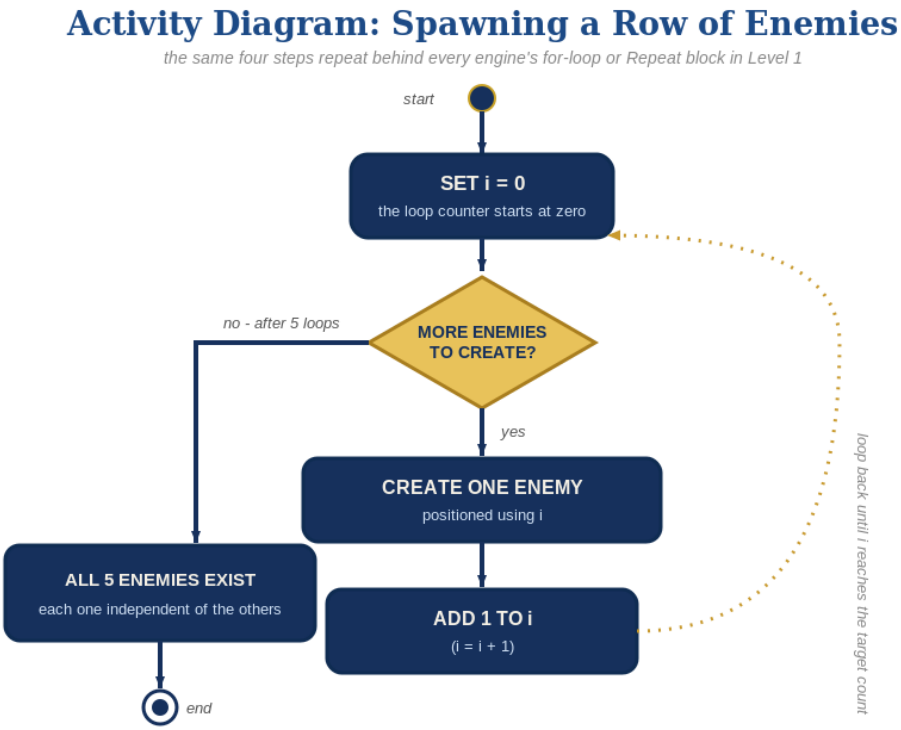
Tip: no new assets this week - you're reusing bullet_yellow.png, sfx_shoot.wav, enemy_invader_red.png, and sfx_hit.wav from your Lesson 3 and 4 Assets folders.

Setup

1. Launch TIC-80 and load your cart from Lesson 4.
2. Press Esc to reach the code editor.

Level 1 - Minimal (MVP)

Several enemies sit on screen in a row, created by a loop instead of placed or spawned one at a time. Shooting any one of them destroys just that one. This is a complete finish for today - reaching this is enough to move on to next week.



Every engine's Level 1 solution runs this same shape - a counter, a repeat check, one creation step, and an increment - even though the exact blocks or code syntax differ. Tracing a spawn bug means finding which of these four steps broke.

The spawn loop, in general - the same counter, repeat check, creation step, and increment behind Level 1 in every engine, whatever the exact blocks or syntax look like.

3. Replace your single enemy_alive/ex/ey variables with an enemies table, populated by a for loop at the top of your code, above the TIC() function.
4. Update the bullets loop so each bullet checks against every enemy in the enemies table (a nested loop), instead of just one fixed position.
5. Update the drawing code at the bottom of TIC() to loop over the enemies table too, only drawing the ones still alive (code below).
6. Press Esc, type run, press Enter, then shoot different enemies to test - each one should disappear independently when hit.

```
enemies={}
for enemy_count=0,4 do
 table.insert(enemies,{ex=20+enemy_count*20,ey=10,alive=true})
end
bullets={}

function TIC()
 cls(0)
 if btn(3) then x=x+1 end
 if btnp(4) then
  table.insert(bullets,{bx=x,by=y})
 end
 for i=#bullets,1,-1 do
  local b=bullets[i]
```

```
  b.by=b.by-2
  local hit=false
  for j=1,#enemies do
   local e=enemies[j]
   if e.alive and b.bx<e.ex+8 and b.bx+8>e.ex and b.by<e.ey+8 and
b.by+8>e.ey then
    e.alive=false
    hit=true
   end
  end
  if hit then
   table.remove(bullets,i)
  elseif b.by<0 then
   table.remove(bullets,i)
  else
   spr(2,b.bx,b.by,0)
  end
 end
 for j=1,#enemies do
  local e=enemies[j]
  if e.alive then spr(3,e.ex,e.ey,0) end
 end
 spr(1,x,y,0)
end
```

Level 2 - Complete (Alpha/Beta)

Every enemy in the row can be individually shot and destroyed without affecting the others, and a bullet that misses every enemy still behaves exactly like it did last week.

- Shoot several different enemies in a row, including some misses - each hit sets exactly the one enemies[j].alive that was actually touched to false, and every other enemy is unaffected.
- The inner enemies loop checks every enemy for every bullet, every frame, so this naturally scales to any number of enemies without extra code - a hit stops mattering for that enemy immediately, since e.alive and ... guards the check the same way it did for the single-enemy version.

Level 3 - Refined (Ready to Publish) - pick any that interest you

- Sound: this already works with no changes - sfx(2) still fires from inside the same hit branch, once per actual hit, no matter which enemy or how many exist.
- Visual: draw a slightly different sprite index or use spr()'s flip argument based on each enemy's position in the table (j), for a bit of visual variety across the row.
- Structure: keep enemy state entirely inside the enemies table (never as loose separate variables again) - every new place that needs an enemy's position or status should read it from that table, the same way the bullets loop does.

If Something's Wrong

- Shooting one enemy removes all of them: confirm the alive flag is being set on enemies[j] specifically (e.alive=false), not on some variable shared by all enemies.
- Enemies don't line up in a row: confirm the spawn loop's ex=20+enemy_count*20 (or your chosen spacing) actually uses the loop variable enemy_count, not a fixed number.

- "attempt to index a nil value" error: confirm enemies={} and its populating for loop run before TIC(), at the very top of the code, same as bullets={}.

Want More?

Student Resource List > TIC-80 section, or tic80.com